HUNDRED DAYS OF CODE
DAY 20

APPLICATION OF ITERATIVE AND DECISION-MAKING CONSTRUCTS TO SHAPES AND PHYSICS USING MODULAR PROGRAMMING IN C

Aim

To develop a menu-driven modular C program to calculate geometrical measurements and physical properties of selected solids using functions, decision-making statements and loops.

The program works with the following solids:

1. Sphere
2. Cylinder
3. Cone
4. Cuboid

The program calculates surface area, volume, base area, mass, weight, gravitational potential energy, pressure, buoyant force and buoyancy behaviour.

Restrictions Given in the Problem

The following rules are followed:

- Only fundamental data types are used.
- Functions use call by value.
- Selection statements are used.
- Loops are used.
- Recursion is not used.
- Arrays are not used.
- Pointer variables are not used.
- Pointer parameters are not used.
- Structures are not used.
- Unions are not used.
- Enumerations are not used.

- typedef is not used.
- Functions from math.h are not used.

All dimensions are entered in metres and densities are entered in kg/m^3. The assignment specifies SI units for all results.

Project Modules

The project is divided into the following files:

File	Purpose
main.c	Displays menus, takes common inputs and displays results
geometry.c	Calculates area, volume and base area
physics.c	Calculates mass, weight, pressure, energy and buoyancy
math_utils.c	Calculates square root using Newton-Raphson method
solid.h	Contains constants and function declarations

This organization follows the module structure given in the assignment.

Constants Given

The following constants are used:

PI = 3.1416

GRAVITY = 9.8

TOLERANCE = 0.000001

Solid choices:

SPHERE = 1

CYLINDER = 2

CONE = 3

CUBOID = 4

Buoyancy results:

FLOATS = -1

HUNDRED DAYS OF CODE
DAY 20

SUSPENDED = 0

SINKS = 1

Algorithms and Pseudocode

Main Program

Algorithm

1. Start the program.
2. Display the main menu.
3. Read the user's choice.
4. Check the choice using switch-case.
5. Perform the selected operation.
6. If the choice is invalid, display an error message.
7. Repeat the menu using a do-while loop.
8. Stop when the user selects Exit.

Pseudocode

START

DO

Display main menu

1. Surface Area
2. Volume
3. Base Area
4. Mass
5. Weight
6. Gravitational Potential Energy

HUNDRED DAYS OF CODE
DAY 20

7. Pressure on Base

8. Buoyant Force

9. Buoyancy Behaviour

10. Exit

Input choice

SWITCH choice

CASE 1:

Calculate surface area

BREAK

CASE 2:

Calculate volume

BREAK

CASE 3:

Calculate base area

BREAK

CASE 4:

Calculate mass

BREAK

CASE 5:

Calculate weight

BREAK

CASE 6:

Calculate potential energy

BREAK

CASE 7:

Calculate pressure

BREAK

CASE 8:

Calculate buoyant force

BREAK

CASE 9:

Determine buoyancy behaviour

BREAK

CASE 10:

Display "Program ended"

BREAK

DEFAULT:

Display "Invalid choice"

END SWITCH

HUNDRED DAYS OF CODE
DAY 20

WHILE choice is not 10

STOP

Input Validation

Every input must be checked immediately.

Rules

- Solid choice must be within the valid range.
- Radius must be greater than zero.
- Length must be greater than zero.
- Breadth must be greater than zero.
- Height must be greater than zero.
- Density must be greater than zero.
- Fluid density must be greater than zero.
- Elevation must be zero or greater.

These validation rules are specified in the assignment.

Pseudocode

INPUT value

WHILE value is invalid

Display "Invalid input"

Ask user to enter the value again

END WHILE

For a positive value:

INPUT value

```
WHILE value <= 0

  Display "Value must be greater than zero"
  INPUT value

END WHILE
```

For elevation:

```
INPUT elevation

WHILE elevation < 0

  Display "Elevation cannot be negative"
  INPUT elevation

END WHILE
```

8. Geometry Operations

8.1 Surface Area of Sphere

Formula

Surface Area = 4 × PI × radius × radius

Algorithm

1. Read radius.
2. Check that radius is greater than zero.
3. Calculate surface area.
4. Return the result.

Pseudocode

HUNDRED DAYS OF CODE
DAY 20

```
START

INPUT radius

WHILE radius <= 0
  Display "Invalid radius"
  INPUT radius
END WHILE

area = 4 × PI × radius × radius

RETURN area

STOP
```

8.2 Volume of Sphere

Formula

Volume = (4/3) × PI × $radius^3$

Pseudocode

```
START

INPUT radius

WHILE radius <= 0
  Display "Invalid radius"
  INPUT radius
END WHILE
```

volume = (4.0 / 3.0) × PI × radius × radius × radius

RETURN volume

STOP

The sphere formulas are given in the assignment.

Cylinder

Surface Area of Cylinder

Formula

Surface Area = 2 × PI × radius × (radius + height)

Pseudocode

START

INPUT radius

INPUT height

Validate radius

Validate height

area = 2 × PI × radius × (radius + height)

RETURN area

STOP

HUNDRED DAYS OF CODE
DAY 20

Volume of Cylinder

Formula

Volume = PI × radius × radius × height

Pseudocode

START

INPUT radius

INPUT height

Validate radius

Validate height

volume = PI × radius × radius × height

RETURN volume

STOP

Cone

Surface Area of Cone

The slant height is required.

Formula

Slant Height = $\sqrt{(radius^2 + height^2)}$

Surface Area = PI × radius × (radius + Slant Height)

Pseudocode

START

HUNDRED DAYS OF CODE
DAY 20

INPUT radius

INPUT height

Validate radius

Validate height

value = radius × radius + height × height

slant_height = calculate_square_root(value)

area = PI × radius × (radius + slant_height)

RETURN area

STOP

The assignment specifically requires the square root to be calculated by calling calculate_square_root().

Volume of Cone

Formula

Volume = (1/3) × PI × $radius^2$ × height

Pseudocode

START

INPUT radius

INPUT height

HUNDRED DAYS OF CODE
DAY 20

Validate radius

Validate height

volume = (1.0 / 3.0) × PI × radius × radius × height

RETURN volume

STOP

Cuboid

Surface Area

Formula

Surface Area =

2 × (length × breadth + breadth × height + length × height)

Pseudocode

START

INPUT length

INPUT breadth

INPUT height

Validate all values

area = 2 × (length × breadth

+ breadth × height

+ length × height)

RETURN area

STOP

Volume

Formula

Volume = length × breadth × height

Pseudocode

START

INPUT length

INPUT breadth

INPUT height

Validate all values

volume = length × breadth × height

RETURN volume

STOP

Base Area

Circular Base

This function is used for cylinders and cones.

Formula

Base Area = PI × $radius^2$

HUNDRED DAYS OF CODE
DAY 20

Pseudocode

START

INPUT radius

Validate radius

area = PI × radius × radius

RETURN area

STOP

Cuboid Base

Formula

Base Area = length × breadth

Pseudocode

START

INPUT length

INPUT breadth

Validate values

area = length × breadth

RETURN area

HUNDRED DAYS OF CODE
DAY 20

STOP

Mass

Formula

Mass = Density × Volume

Algorithm

1. Select the solid.
2. Enter density.
3. Enter the required dimensions.
4. Calculate volume using the appropriate function.
5. Multiply density by volume.
6. Return mass.

Pseudocode

START

Input solid type

Input density

Validate density

IF solid is sphere

  Input radius

  volume = sphere_volume(radius)

ELSE IF solid is cylinder

  Input radius and height

  volume = cylinder_volume(radius, height)

ELSE IF solid is cone

Input radius and height

volume = cone_volume(radius, height)

ELSE IF solid is cuboid

Input length, breadth and height

volume = cuboid_volume(length, breadth, height)

END IF

mass = density × volume

RETURN mass

STOP

The assignment specifies that mass is obtained using the appropriate volume function and mass = density × volume.

Weight

Formula

Weight = Mass × GRAVITY

Pseudocode

START

mass = calculate_mass(density)

weight = mass × GRAVITY

RETURN weight

STOP

The value of gravity is 9.8 m/s^2.

15. Gravitational Potential Energy

Formula

Potential Energy = Mass × GRAVITY × Elevation

Pseudocode

START

Input elevation

WHILE elevation < 0

  Display "Elevation cannot be negative"

  Input elevation

END WHILE

mass = calculate_mass(density)

energy = mass × GRAVITY × elevation

RETURN energy

STOP

The assignment specifies that mass is obtained first and then potential energy is calculated using mass, gravity and elevation.

HUNDRED DAYS OF CODE
DAY 20

Pressure on the Base

Pressure is calculated only for:

- Cylinder
- Cone
- Cuboid

A sphere is not used because its ideal contact area is not defined in the assignment.

Formula

Pressure = (Density × Volume × GRAVITY) / Contact Area

Pseudocode

START

Input solid type

Input density

Validate density

IF solid is cylinder

  Input radius

  Input height

  contact_area = circular_base_area(radius)

  volume = cylinder_volume(radius, height)

ELSE IF solid is cone

HUNDRED DAYS OF CODE
DAY 20

```
    Input radius
    Input height

    contact_area = circular_base_area(radius)
    volume = cone_volume(radius, height)

ELSE IF solid is cuboid

    Input length
    Input breadth
    Input height

    contact_area = cuboid_base_area(length, breadth)
    volume = cuboid_volume(length, breadth, height)

ELSE

    Display "Pressure cannot be calculated for sphere"

END IF

pressure = (density × volume × GRAVITY) / contact_area

RETURN pressure

STOP
```

HUNDRED DAYS OF CODE
DAY 20

Buoyant Force

Formula

Buoyant Force = Fluid Density × Volume × GRAVITY

Pseudocode

START

Input solid type

Input fluid density

Validate fluid density

IF solid is sphere

  Input radius

  volume = sphere_volume(radius)

ELSE IF solid is cylinder

  Input radius and height

  volume = cylinder_volume(radius, height)

ELSE IF solid is cone

  Input radius and height

  volume = cone_volume(radius, height)

ELSE IF solid is cuboid

  Input length, breadth and height

  volume = cuboid_volume(length, breadth, height)

END IF

force = fluid_density × volume × GRAVITY

RETURN force

STOP

This follows the assignment's specified buoyant-force formula.

Buoyancy Behaviour

The solid's density is compared with the fluid's density.

Rules

If solid density > fluid density

Solid sinks

If solid density < fluid density

Solid floats

If solid density = fluid density

Solid remains suspended

Pseudocode

START

Input solid density

Input fluid density

Validate both densities

HUNDRED DAYS OF CODE
DAY 20

IF solid density > fluid density

  result = SINKS

ELSE IF solid density < fluid density

  result = FLOATS

ELSE

  result = SUSPENDED

END IF

RETURN result

STOP

Newton-Raphson Square Root

Since sqrt() from math.h cannot be used, the program calculates square root using the Newton-Raphson method.

Algorithm

1. If the number is negative, return -1.
2. If the number is zero, return 0.
3. Set old_value = number.
4. Calculate a new approximation.
5. Calculate the difference.
6. Make the difference positive if required.
7. Continue until the difference becomes smaller than the tolerance.
8. Return the final value.

Formula

HUNDRED DAYS OF CODE
DAY 20

new_value = 0.5 × (old_value + number / old_value)

Pseudocode

```
START

INPUT number

IF number < 0
   RETURN -1

IF number = 0
   RETURN 0

old_value = number

DO

   new_value = 0.5 × (old_value + number / old_value)

   difference = new_value - old_value

   IF difference < 0
      difference = -difference
   END IF

   old_value = new_value

WHILE difference >= TOLERANCE
```

HUNDRED DAYS OF CODE
DAY 20

RETURN new_value

STOP

Test Case Table

Main Menu

Test Case	Input	Expected Result	Type
1	1	Surface Area menu opens	Valid
2	5	Weight operation opens	Valid
3	10	Program exits	Boundary
4	0	Invalid choice message	Invalid
5	11	Invalid choice message	Invalid

Sphere

Test Case	Radius	Expected Result	Type
1	5	Surface area calculated	Valid
2	5	Volume calculated	Valid
3	0	Error message	Boundary/Invalid
4	-2	Error message	Invalid

Cylinder

Test Case	Radius	Height	Expected Result	Type
1	2	5	Area calculated	Valid
2	2	5	Volume calculated	Valid

HUNDRED DAYS OF CODE
DAY 20

Test Case	Radius	Height	Expected Result	Type
3	0	5	Error message	Boundary/Invalid
4	2	0	Error message	Boundary/Invalid
5	-2	5	Error message	Invalid

Cone

Test Case	Radius	Height	Expected Result	Type
1	3	4	Area calculated	Valid
2	3	4	Volume calculated	Valid
3	0	4	Error message	Boundary/Invalid
4	3	0	Error message	Boundary/Invalid
5	-3	4	Error message	Invalid

Cuboid

Test Case	Length	Breadth	Height	Expected Result	Type
1	2	3	4	Area calculated	Valid
2	2	3	4	Volume calculated	Valid
3	0	3	4	Error message	Boundary/Invalid
4	2	0	4	Error message	Boundary/Invalid
5	2	3	0	Error message	Boundary/Invalid
6	-2	3	4	Error message	Invalid

Density and Elevation

HUNDRED DAYS OF CODE
DAY 20

Test Case	Input	Expected Result	Type
1	Density = 1000	Accepted	Valid
2	Density = 0	Error message	Boundary/Invalid
3	Density = -100	Error message	Invalid
4	Elevation = 10	Accepted	Valid
5	Elevation = 0	Accepted	Boundary
6	Elevation = -5	Error message	Invalid

Buoyancy

Test Case	Solid Density	Fluid Density	Expected Result	Type
1	500	1000	FLOATS	Valid
2	1000	1000	SUSPENDED	Boundary
3	1200	1000	SINKS	Valid
4	0	1000	Error message	Invalid
5	1000	0	Error message	Invalid

Expected Menu

========== SOLID CALCULATOR ==========

1. Surface Area
2. Volume
3. Base Area
4. Mass
5. Weight
6. Gravitational Potential Energy
7. Pressure on Base

8. Buoyant Force

9. Buoyancy Behaviour

10. Exit

Enter your choice:

Complete C Program

File 1: solid.h

```
#ifndef SOLID_H

#define SOLID_H


#define PI 3.1416

#define GRAVITY 9.8

#define TOLERANCE 0.000001


#define SPHERE 1

#define CYLINDER 2

#define CONE 3

#define CUBOID 4


#define FLOATS -1

#define SUSPENDED 0

#define SINKS 1


/* Geometry functions */

double sphere_area(double radius);

double sphere_volume(double radius);
```

```
double cylinder_area(double radius, double height);
double cylinder_volume(double radius, double height);

double cone_area(double radius, double height);
double cone_volume(double radius, double height);

double cuboid_area(double length, double breadth, double height);
double cuboid_volume(double length, double breadth, double height);

double circular_base_area(double radius);
double cuboid_base_area(double length, double breadth);

/* Physics functions */
double calculate_mass(double density);
double calculate_weight();
double potential_energy(double elevation);
double calculate_pressure(double density);
double buoyant_force(double fluid_density);
int determine_buoyancy(double solid_density, double fluid_density);

/* Math function */
double calculate_square_root(double number);

#endif
```

File 2: math_utils.c

HUNDRED DAYS OF CODE
DAY 20

```
#include "solid.h"


double calculate_square_root(double number)
{
   double old_value;
   double new_value;
   double difference;

   if (number < 0)
   {
      return -1;
   }

   if (number == 0)
   {
      return 0;
   }

   old_value = number;

   do
   {
      new_value = 0.5 * (old_value + number / old_value);

      difference = new_value - old_value;

      if (difference < 0)
```

```
    {
        difference = -difference;
    }

    old_value = new_value;

  } while (difference >= TOLERANCE);

  return new_value;
}
```

File 3: geometry.c

```
#include "solid.h"

double sphere_area(double radius)
{
  return 4.0 * PI * radius * radius;
}

double sphere_volume(double radius)
{
  return (4.0 / 3.0) * PI * radius * radius * radius;
}

double cylinder_area(double radius, double height)
{
  return 2.0 * PI * radius * (radius + height);
}
```

HUNDRED DAYS OF CODE
DAY 20

```
double cylinder_volume(double radius, double height)
{
   return PI * radius * radius * height;
}

double cone_area(double radius, double height)
{
   double slant_height;
   double value;

   value = radius * radius + height * height;

   slant_height = calculate_square_root(value);

   return PI * radius * (radius + slant_height);
}

double cone_volume(double radius, double height)
{
   return (1.0 / 3.0) * PI * radius * radius * height;
}

double cuboid_area(double length, double breadth, double height)
{
   return 2.0 * (length * breadth
           + breadth * height
```

```
        + length * height);
}

double cuboid_volume(double length, double breadth, double height)
{
   return length * breadth * height;
}

double circular_base_area(double radius)
{
   return PI * radius * radius;
}

double cuboid_base_area(double length, double breadth)
{
   return length * breadth;
}
```

File 4: physics.c

```
#include <stdio.h>
#include "solid.h"

double calculate_mass(double density)
{
   int type;
   double radius;
   double height;
```

HUNDRED DAYS OF CODE
DAY 20

```
double length;
double breadth;
double volume;

do
{
   printf("\nSelect Solid:\n");
   printf("1. Sphere\n");
   printf("2. Cylinder\n");
   printf("3. Cone\n");
   printf("4. Cuboid\n");
   printf("Enter choice: ");
   scanf("%d", &type);

   if (type < 1 || type > 4)
   {
      printf("Invalid solid choice.\n");
   }

} while (type < 1 || type > 4);

if (type == SPHERE)
{
   do
   {
      printf("Enter radius: ");
      scanf("%lf", &radius);
```

```
        if (radius <= 0)
            printf("Radius must be greater than zero.\n");

    } while (radius <= 0);

    volume = sphere_volume(radius);
}

else if (type == CYLINDER)
{
    do
    {
        printf("Enter radius: ");
        scanf("%lf", &radius);

        if (radius <= 0)
            printf("Radius must be greater than zero.\n");

    } while (radius <= 0);

    do
    {
        printf("Enter height: ");
        scanf("%lf", &height);

        if (height <= 0)
```

```
        printf("Height must be greater than zero.\n");

    } while (height <= 0);

    volume = cylinder_volume(radius, height);
}

else if (type == CONE)
{
    do
    {
        printf("Enter radius: ");
        scanf("%lf", &radius);

        if (radius <= 0)
            printf("Radius must be greater than zero.\n");

    } while (radius <= 0);

    do
    {
        printf("Enter height: ");
        scanf("%lf", &height);

        if (height <= 0)
            printf("Height must be greater than zero.\n");
```

```
    } while (height <= 0);

    volume = cone_volume(radius, height);
}

else
{
    do
    {
        printf("Enter length: ");
        scanf("%lf", &length);

        if (length <= 0)
            printf("Length must be greater than zero.\n");

    } while (length <= 0);

    do
    {
        printf("Enter breadth: ");
        scanf("%lf", &breadth);

        if (breadth <= 0)
            printf("Breadth must be greater than zero.\n");

    } while (breadth <= 0);
```

```
    do
    {
        printf("Enter height: ");
        scanf("%lf", &height);

        if (height <= 0)
            printf("Height must be greater than zero.\n");

    } while (height <= 0);

    volume = cuboid_volume(length, breadth, height);
  }

  return density * volume;
}



double calculate_weight()
{
  double density;
  double mass;

  do
  {
    printf("Enter density (kg/m^3): ");
    scanf("%lf", &density);
```

```
    if (density <= 0)
        printf("Density must be greater than zero.\n");

  } while (density <= 0);

  mass = calculate_mass(density);

  return mass * GRAVITY;
}



double potential_energy(double elevation)
{
  double density;
  double mass;

  do
  {
    printf("Enter density (kg/m^3): ");
    scanf("%lf", &density);

    if (density <= 0)
        printf("Density must be greater than zero.\n");

  } while (density <= 0);

  mass = calculate_mass(density);
```

```
    return mass * GRAVITY * elevation;
}



double calculate_pressure(double density)
{
    int type;
    double radius;
    double height;
    double length;
    double breadth;
    double contact_area;
    double volume;

    do
    {
        printf("\nSelect Solid:\n");
        printf("1. Cylinder\n");
        printf("2. Cone\n");
        printf("3. Cuboid\n");
        printf("Enter choice: ");
        scanf("%d", &type);

        if (type < 1 || type > 3)
            printf("Invalid choice. Sphere is not supported for pressure.\n");
```

HUNDRED DAYS OF CODE
DAY 20

```
} while (type < 1 || type > 3);


do
{
   printf("Enter density (kg/m^3): ");
   scanf("%lf", &density);

   if (density <= 0)
      printf("Density must be greater than zero.\n");

} while (density <= 0);

if (type == 1)
{
   do
   {
      printf("Enter radius: ");
      scanf("%lf", &radius);

      if (radius <= 0)
         printf("Radius must be greater than zero.\n");

   } while (radius <= 0);

   do
   {
      printf("Enter height: ");
```

```
    scanf("%lf", &height);

    if (height <= 0)
      printf("Height must be greater than zero.\n");

  } while (height <= 0);

  contact_area = circular_base_area(radius);
  volume = cylinder_volume(radius, height);
}

else if (type == 2)
{
  do
  {
    printf("Enter radius: ");
    scanf("%lf", &radius);

    if (radius <= 0)
      printf("Radius must be greater than zero.\n");

  } while (radius <= 0);

  do
  {
    printf("Enter height: ");
    scanf("%lf", &height);
```

```
    if (height <= 0)
        printf("Height must be greater than zero.\n");

  } while (height <= 0);

  contact_area = circular_base_area(radius);
  volume = cone_volume(radius, height);
}

else
{
  do
  {
    printf("Enter length: ");
    scanf("%lf", &length);

    if (length <= 0)
        printf("Length must be greater than zero.\n");

  } while (length <= 0);

  do
  {
    printf("Enter breadth: ");
    scanf("%lf", &breadth);
```

```
        if (breadth <= 0)
            printf("Breadth must be greater than zero.\n");

    } while (breadth <= 0);

    do
    {
        printf("Enter height: ");
        scanf("%lf", &height);

        if (height <= 0)
            printf("Height must be greater than zero.\n");

    } while (height <= 0);

    contact_area = cuboid_base_area(length, breadth);
    volume = cuboid_volume(length, breadth, height);
  }

  return (density * volume * GRAVITY) / contact_area;
}


double buoyant_force(double fluid_density)
{
  int type;
  double radius;
```

HUNDRED DAYS OF CODE
DAY 20

```
double height;
double length;
double breadth;
double volume;

do
{
   printf("\nSelect Solid:\n");
   printf("1. Sphere\n");
   printf("2. Cylinder\n");
   printf("3. Cone\n");
   printf("4. Cuboid\n");
   printf("Enter choice: ");
   scanf("%d", &type);

   if (type < 1 || type > 4)
      printf("Invalid solid choice.\n");

} while (type < 1 || type > 4);

if (type == SPHERE)
{
   do
   {
      printf("Enter radius: ");
      scanf("%lf", &radius);
```

```
    if (radius <= 0)
        printf("Radius must be greater than zero.\n");

  } while (radius <= 0);

  volume = sphere_volume(radius);
}

else if (type == CYLINDER)
{
  do
  {
    printf("Enter radius: ");
    scanf("%lf", &radius);

    if (radius <= 0)
        printf("Radius must be greater than zero.\n");

  } while (radius <= 0);

  do
  {
    printf("Enter height: ");
    scanf("%lf", &height);

    if (height <= 0)
        printf("Height must be greater than zero.\n");
```

```
    } while (height <= 0);

    volume = cylinder_volume(radius, height);
}

else if (type == CONE)
{
    do
    {
        printf("Enter radius: ");
        scanf("%lf", &radius);

        if (radius <= 0)
            printf("Radius must be greater than zero.\n");

    } while (radius <= 0);

    do
    {
        printf("Enter height: ");
        scanf("%lf", &height);

        if (height <= 0)
            printf("Height must be greater than zero.\n");

    } while (height <= 0);
```

```
    volume = cone_volume(radius, height);
}

else
{
    do
    {
        printf("Enter length: ");
        scanf("%lf", &length);

        if (length <= 0)
            printf("Length must be greater than zero.\n");

    } while (length <= 0);

    do
    {
        printf("Enter breadth: ");
        scanf("%lf", &breadth);

        if (breadth <= 0)
            printf("Breadth must be greater than zero.\n");

    } while (breadth <= 0);

    do
```

```
    {
        printf("Enter height: ");
        scanf("%lf", &height);

        if (height <= 0)
            printf("Height must be greater than zero.\n");

    } while (height <= 0);

    volume = cuboid_volume(length, breadth, height);
  }

  return fluid_density * volume * GRAVITY;
}



int determine_buoyancy(double solid_density, double fluid_density)
{
  if (solid_density > fluid_density)
  {
    return SINKS;
  }
  else if (solid_density < fluid_density)
  {
    return FLOATS;
  }
  else
```

```
  {
    return SUSPENDED;
  }
}
```

File 5: main.c

```
#include <stdio.h>
#include "solid.h"

int main()
{
  int choice;
  int type;
  int result;

  double radius;
  double height;
  double length;
  double breadth;
  double density;
  double fluid_density;
  double elevation;
  double answer;

  do
  {
    printf("\n====================================\n");
```

HUNDRED DAYS OF CODE
DAY 20

```
printf("      SOLID CALCULATOR\n");

printf("====================================\n");


printf("1. Surface Area\n");

printf("2. Volume\n");

printf("3. Base Area\n");

printf("4. Mass\n");

printf("5. Weight\n");

printf("6. Gravitational Potential Energy\n");

printf("7. Pressure on the Base\n");

printf("8. Buoyant Force\n");

printf("9. Buoyancy Behaviour\n");

printf("10. Exit\n");


printf("\nEnter your choice: ");

scanf("%d", &choice);


switch (choice)

{

   case 1:


      printf("\nSelect Solid:\n");

      printf("1. Sphere\n");

      printf("2. Cylinder\n");

      printf("3. Cone\n");

      printf("4. Cuboid\n");

      printf("Enter choice: ");
```

HUNDRED DAYS OF CODE
DAY 20

```
scanf("%d", &type);


if (type == SPHERE)
{
   do
   {
      printf("Enter radius: ");
      scanf("%lf", &radius);


      if (radius <= 0)
         printf("Radius must be greater than zero.\n");


   } while (radius <= 0);


   answer = sphere_area(radius);
}


else if (type == CYLINDER)
{
   do
   {
      printf("Enter radius: ");
      scanf("%lf", &radius);


      if (radius <= 0)
         printf("Radius must be greater than zero.\n");
```

```
    } while (radius <= 0);

    do
    {
        printf("Enter height: ");
        scanf("%lf", &height);

        if (height <= 0)
            printf("Height must be greater than zero.\n");

    } while (height <= 0);

    answer = cylinder_area(radius, height);
}

else if (type == CONE)
{
    do
    {
        printf("Enter radius: ");
        scanf("%lf", &radius);

        if (radius <= 0)
            printf("Radius must be greater than zero.\n");

    } while (radius <= 0);
```

```
    do
    {
        printf("Enter height: ");
        scanf("%lf", &height);

        if (height <= 0)
            printf("Height must be greater than zero.\n");

    } while (height <= 0);

    answer = cone_area(radius, height);
}

else if (type == CUBOID)
{
    do
    {
        printf("Enter length: ");
        scanf("%lf", &length);

        if (length <= 0)
            printf("Length must be greater than zero.\n");

    } while (length <= 0);

    do
    {
```

```
        printf("Enter breadth: ");
        scanf("%lf", &breadth);

        if (breadth <= 0)
            printf("Breadth must be greater than zero.\n");

    } while (breadth <= 0);

    do
    {
        printf("Enter height: ");
        scanf("%lf", &height);

        if (height <= 0)
            printf("Height must be greater than zero.\n");

    } while (height <= 0);

    answer = cuboid_area(length, breadth, height);
}

else
{
    printf("Invalid solid choice.\n");
    break;
}
```

```
   printf("Surface Area = %.4lf m^2\n", answer);
   break;



case 2:


   printf("\nSelect Solid:\n");
   printf("1. Sphere\n");
   printf("2. Cylinder\n");
   printf("3. Cone\n");
   printf("4. Cuboid\n");
   printf("Enter choice: ");
   scanf("%d", &type);


   if (type == SPHERE)
   {
      do
      {
         printf("Enter radius: ");
         scanf("%lf", &radius);


         if (radius <= 0)
            printf("Radius must be greater than zero.\n");


      } while (radius <= 0);


      answer = sphere_volume(radius);
```

```
}

else if (type == CYLINDER)
{
   do
   {
      printf("Enter radius: ");
      scanf("%lf", &radius);

      if (radius <= 0)
         printf("Radius must be greater than zero.\n");

   } while (radius <= 0);

   do
   {
      printf("Enter height: ");
      scanf("%lf", &height);

      if (height <= 0)
         printf("Height must be greater than zero.\n");

   } while (height <= 0);

   answer = cylinder_volume(radius, height);
}
```

HUNDRED DAYS OF CODE
DAY 20

```
else if (type == CONE)
{
   do
   {
      printf("Enter radius: ");
      scanf("%lf", &radius);

      if (radius <= 0)
         printf("Radius must be greater than zero.\n");

   } while (radius <= 0);

   do
   {
      printf("Enter height: ");
      scanf("%lf", &height);

      if (height <= 0)
         printf("Height must be greater than zero.\n");

   } while (height <= 0);

   answer = cone_volume(radius, height);
}

else if (type == CUBOID)
{
```

HUNDRED DAYS OF CODE
DAY 20

```
do
{
   printf("Enter length: ");
   scanf("%lf", &length);

   if (length <= 0)
      printf("Length must be greater than zero.\n");

} while (length <= 0);

do
{
   printf("Enter breadth: ");
   scanf("%lf", &breadth);

   if (breadth <= 0)
      printf("Breadth must be greater than zero.\n");

} while (breadth <= 0);

do
{
   printf("Enter height: ");
   scanf("%lf", &height);

   if (height <= 0)
      printf("Height must be greater than zero.\n");
```

```
    } while (height <= 0);

    answer = cuboid_volume(length, breadth, height);
  }

  else
  {
    printf("Invalid solid choice.\n");
    break;
  }

  printf("Volume = %.4lf m^3\n", answer);
  break;


case 3:

  printf("\nSelect Solid:\n");
  printf("1. Cylinder\n");
  printf("2. Cone\n");
  printf("3. Cuboid\n");
  printf("Enter choice: ");
  scanf("%d", &type);

  if (type == 1 || type == 2)
  {
```

```
    do
    {
        printf("Enter radius: ");
        scanf("%lf", &radius);

        if (radius <= 0)
            printf("Radius must be greater than zero.\n");

    } while (radius <= 0);

    answer = circular_base_area(radius);
}

else if (type == 3)
{
    do
    {
        printf("Enter length: ");
        scanf("%lf", &length);

        if (length <= 0)
            printf("Length must be greater than zero.\n");

    } while (length <= 0);

    do
    {
```

```
        printf("Enter breadth: ");
        scanf("%lf", &breadth);

        if (breadth <= 0)
            printf("Breadth must be greater than zero.\n");

    } while (breadth <= 0);

    answer = cuboid_base_area(length, breadth);
}

else
{
    printf("Invalid choice. Sphere does not have a defined base area.\n");
    break;
}

printf("Base Area = %.4lf m^2\n", answer);
break;


case 4:

do
{
    printf("Enter density (kg/m^3): ");
    scanf("%lf", &density);
```

```
        if (density <= 0)
            printf("Density must be greater than zero.\n");

    } while (density <= 0);

    answer = calculate_mass(density);

    printf("Mass = %.4lf kg\n", answer);
    break;


case 5:

    answer = calculate_weight();

    printf("Weight = %.4lf N\n", answer);
    break;


case 6:

    do
    {
        printf("Enter elevation (m): ");
        scanf("%lf", &elevation);
```

```
    if (elevation < 0)
        printf("Elevation cannot be negative.\n");

  } while (elevation < 0);

  answer = potential_energy(elevation);

  printf("Potential Energy = %.4lf J\n", answer);
  break;


case 7:

  do
  {
    printf("Enter density (kg/m^3): ");
    scanf("%lf", &density);

    if (density <= 0)
        printf("Density must be greater than zero.\n");

  } while (density <= 0);

  answer = calculate_pressure(density);

  printf("Pressure = %.4lf Pa\n", answer);
  break;
```

HUNDRED DAYS OF CODE
DAY 20

```
case 8:

  do
  {
    printf("Enter fluid density (kg/m^3): ");
    scanf("%lf", &fluid_density);

    if (fluid_density <= 0)
      printf("Fluid density must be greater than zero.\n");

  } while (fluid_density <= 0);

  answer = buoyant_force(fluid_density);

  printf("Buoyant Force = %.4lf N\n", answer);
  break;


case 9:

  do
  {
    printf("Enter solid density (kg/m^3): ");
    scanf("%lf", &density);
```

```
    if (density <= 0)
        printf("Density must be greater than zero.\n");

} while (density <= 0);

do
{
    printf("Enter fluid density (kg/m^3): ");
    scanf("%lf", &fluid_density);

    if (fluid_density <= 0)
        printf("Fluid density must be greater than zero.\n");

} while (fluid_density <= 0);

result = determine_buoyancy(density, fluid_density);

if (result == SINKS)
    printf("The solid sinks.\n");

else if (result == FLOATS)
    printf("The solid floats.\n");

else
    printf("The solid remains suspended.\n");

break;
```

```
        case 10:

            printf("\nProgram ended.\n");
            break;



        default:

            printf("\nInvalid choice. Please enter a number from 1 to 10.\n");
    }

  } while (choice != 10);

  return 0;
}
```

Compilation and execution

The four C files can be compiled together using:

gcc main.c geometry.c physics.c math_utils.c -o solid

Then run the program using:

./solid

HUNDRED DAYS OF CODE
DAY 20

```
========== SOLID PROPERTY CALCULATOR ==========
1. Surface area
2. Volume
3. Base Area
4. Mass
5. Weight
6. Gravitational potential energy
7. Pressure on the base
8. Buoyant force
9. Buoyancy behavior
10. Exit
Enter your choice: 1

Select solid:
1. Sphere
2. Cylinder
3. Cone
4. Cuboid
Enter choice: 3
Enter radius (m): 3
Enter height (m): 4
Total surface area = 75.398400 m^2
```

```
========== SOLID PROPERTY CALCULATOR ==========
1. Surface area
2. Volume
3. Base Area
4. Mass
5. Weight
6. Gravitational potential energy
7. Pressure on the base
8. Buoyant force
9. Buoyancy behavior
10. Exit
Enter your choice: 9
Enter solid density (kg/m^3): 1200
Enter fluid density (kg/m^3): 1000
The solid sinks.
```

HUNDRED DAYS OF CODE
DAY 20

Conclusion

The project successfully demonstrates the use of modular programming in C for calculating geometrical and physical properties of different solids.

Functions are used to divide the program into smaller parts. switch-case is used for menu selection, while if-else is used for decision-making. do-while loops are used for menu repetition and input validation.

The program calculates surface area, volume, base area, mass, weight, gravitational potential energy, pressure and buoyant force. It also determines whether a solid floats, sinks or remains suspended.

The Newton-Raphson method is used to calculate square root without using functions from math.h. Therefore, the main requirements of the given assignment are implemented using basic C programming concepts.

.....